SAMPARK / UNDERSTAND THE WHOLE SYSTEM

Every expression.
Every building block.

A plain-English map of the actual project: what we built, which open-source tools help, which models run remotely, and how a small expression becomes a confirmed voice.

01 / CHOOSE

Give permission

Choose speech, gesture, or lip input.

02 / RECOGNIZE

Catch the pieces

Audio and movement become words or hints.

03 / UNDERSTAND

Add context

Include the partner's question and your vocabulary.

04 / SUGGEST

Find the wording

A model proposes whole sentences; our rules check them.

05 / APPROVE

Keep your choice

Review/edit or use the optional cancelable auto-say.

06 / CONNECT

Say the message

Play approved voice, with an avatar when available.

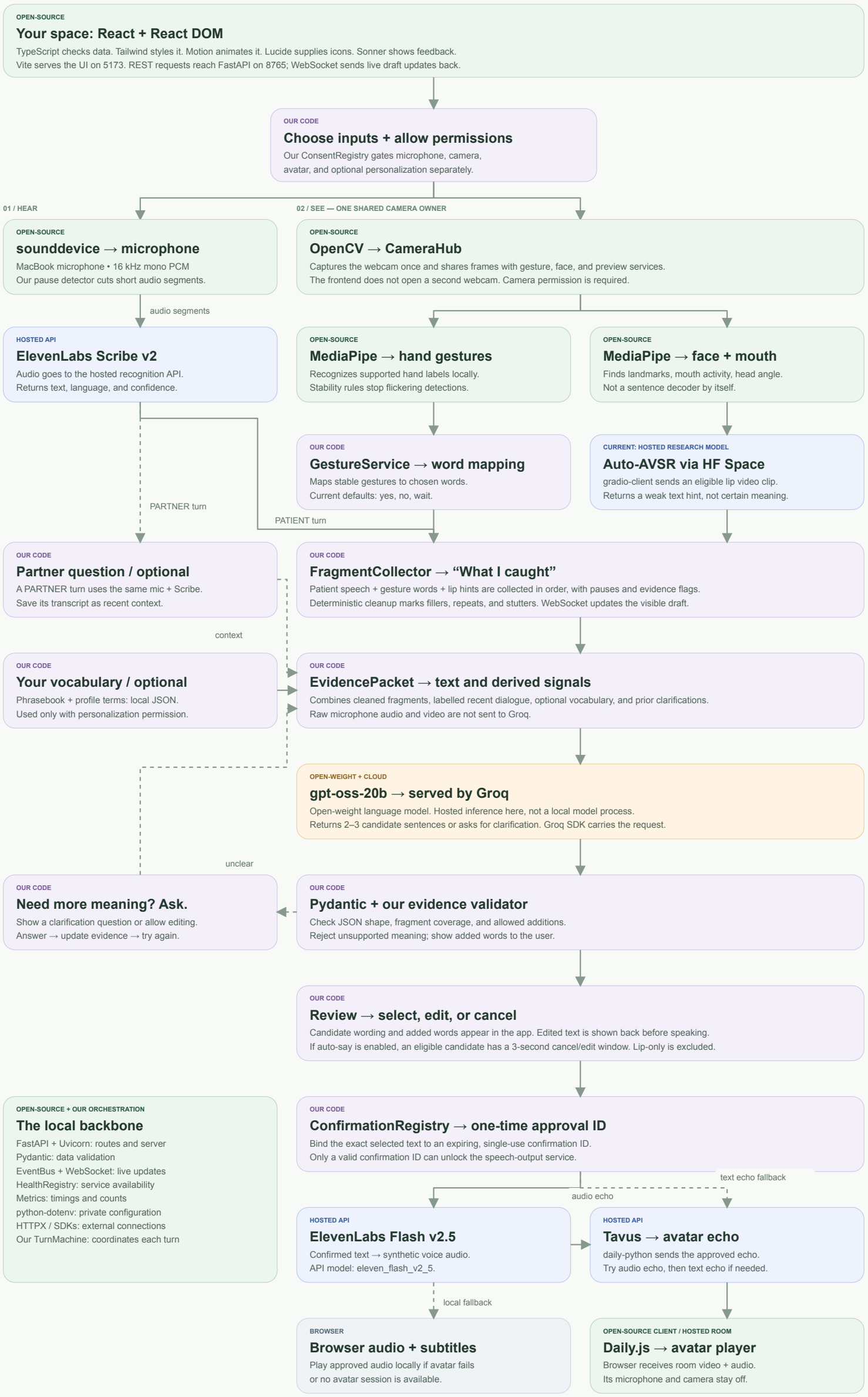
Open-source does not mean everything runs offline.
The app uses open-source libraries, an open-weight language model hosted by Groq, and hosted recognition/voice/avatar APIs. An open-source SDK is the connector, not ownership of the service or its model.

Open-source library · Hosted API · Open-weight + hosted · Our code · Browser API

SAMPARK / THE COMPLETE FLOW

Small expressions. One message.

Solid arrows = normal flow. Dashed arrows = optional context, clarification, or fallback.



WHAT LEAVES THE LAPTOP?

Audio → ElevenLabs Scribe. Text evidence → Groq. Lip clip → HF Space when used.

Confirmed text/audio → ElevenLabs / Tavus / Daily for output. The entire system is not offline.

Optional local lip path: Auto-AVSR + PyTorch instead of the hosted Space. Code and model licences are separate.

Based on repository code + non-secret configuration • 1 November 2026 • no credentials or patient data included

Where each kind of data goes.

Stays on the laptop

Device control, MediaPipe face/gesture tracking, fragment cleanup, validation, confirmation, and optional vocabulary files.

Uses cloud services

Audio goes to Scribe; text evidence goes to Groq; eligible lip video goes to the HF Space; approved voice/avatar output uses ElevenLabs and Tavus/Daily.

Optional local lip model

Auto-AVSR + PyTorch can replace the hosted Space when source and a compatible checkpoint are installed. Code and weight licences are separate.

Every component, in one place.

Direct dependencies, explicitly supported optional tools, and our orchestration modules. Included-but-secondary components are labelled; this is not an exhaustive transitive SBOM.

AREA	COMPONENT	TYPE	USED WHEN	WHAT IT DOES	WHERE IN CODE	LICENCE / TERMS
Experience	**React + React DOM**	Open-source	Live	Build and update the six application screens.	`f_new/src/App.tsx; f_new/src/components/ps16-app.tsx`	MIT
Experience	**TypeScript**	Open-source	Build-time	Checks frontend data shapes and catches coding errors.	`f_new/tsconfig.json; f_new/src/lib/types.ts`	Apache-2.0
Experience	**Vite + React plugin**	Open-source	Dev/build	Serves the frontend on 5173, builds production files, and proxies /backend requests to Python.	`f_new/vite.config.ts`	MIT
Experience	**Tailwind CSS + @tailwindcss/vite**	Open-source	Build-time	Turns styling utilities and theme tokens into the app's CSS.	`f_new/src/app-theme.css; f_new/vite.config.ts`	MIT
Experience	**Motion**	Open-source	Live	Animates screen transitions, fragments, and confirmation views; honors reduced motion.	`f_new/src/components/ps16-app.tsx; f_new/src/components/screens/message.tsx`	MIT
Experience	**Lucide React**	Open-source	Live	Provides microphone, camera, permission, and navigation icons.	`f_new/src/components/`	ISC
Experience	**Sonner**	Open-source	Live	Shows success messages and actionable errors.	`f_new/src/lib/backend.tsx; f_new/src/components/ps16-app.tsx`	MIT
Experience	**clsx + tailwind-merge**	Open-source	Live	Combines conditional CSS classes and resolves conflicting Tailwind classes.	`f_new/src/lib/utils.ts`	MIT
Experience	**tw-animate-css**	Open-source	Build-time	Supplies reusable CSS animation utilities.	`f_new/src/app-theme.css`	MIT
Experience	**Base UI + class-variance-authority**	Open-source	Included wrapper	Accessible button primitive and variant helper in the generic button wrapper; active screens mainly use the custom UI kit.	`f_new/src/components/ui/button.tsx`	MIT / Apache-2.0
Experience	**@types/react + @types/react-dom**	Open-source	Build-time	React type declarations used by TypeScript.	`f_new/package.json`	MIT
Backend	**FastAPI + Starlette**	Open-source	Live / local	Expose permission, turn, voice, context, avatar, and WebSocket endpoints on 8765.	`app/main.py; app/api/`	MIT / BSD-3-Clause
Backend	**Uvicorn + websockets**	Open-source	Live / local	Run the Python web server and carry live events to the browser.	`app/main.py; app/api/ws.py`	BSD-3-Clause
Backend	**Pydantic**	Open-source	Live / local	Validate API requests, evidence packets, candidates, and model JSON.	`app/schemas.py; app/services/fusion.py`	MIT
Backend	**python-dotenv**	Open-source	Startup / local	Reads local configuration and API credentials from .env. Secrets are not sent to the frontend.	`app/config.py`	BSD-3-Clause
Backend	**HTTPX**	Open-source	Live	Makes external HTTP requests, including Tavus operations and service checks.	`app/services/avatar.py; scripts/check_env.py`	BSD-3-Clause

AREA	COMPONENT	TYPE	USED WHEN	WHAT IT DOES	WHERE IN CODE	LICENCE / TERMS
Input	**sounddevice / PortAudio**	Open-source	Live / local	Records the selected laptop microphone at 16 kHz mono PCM. Current device: MacBook Air Microphone.	`app/services/scribe.py`	MIT / see PortAudio distribution
Input	**NumPy**	Open-source	Live / local	Computes audio levels and manipulates image arrays and landmark coordinates.	`app/services/scribe.py`; `app/services/face.py`; `app/services/lip.py`	BSD-3-Clause
Input	**OpenCV / opencv-contrib-python**	Open-source	Live / local	Owns webcam capture through CameraHub; prepares preview images and lip clips.	`app/services/camera.py`; `app/services/lip.py`	Apache-2.0
Input	**MediaPipe Face Landmarker**	Open-source toolkit + separate model asset	Live / local	Finds face landmarks, head direction, and mouth movement. It does not produce lip-reading sentences by itself.	`app/services/face.py`; `assets/face_landmarker.task`	Toolkit Apache-2.0; model-asset licence not confirmed in repository audit
Input	**MediaPipe Gesture Recognizer**	Open-source toolkit + separate model asset	Live / local	Recognizes supported hand labels, then custom rules map stable labels to words such as yes, no, or wait.	`app/services/gesture.py`; `calibration/default.json`	Toolkit Apache-2.0; model-asset licence not confirmed in repository audit
Recognition	**ElevenLabs SDK**	Open-source client	Live	Connects Python to ElevenLabs speech recognition and voice generation. The SDK licence does not make the hosted models open-source.	`app/services/scribe.py`; `app/services/tts.py`	MIT
Recognition	**ElevenLabs Scribe v2**	Hosted service	Live / cloud	Transcribes microphone audio segments into words after pauses. Model ID: scribe_v2.	`app/services/scribe.py`	Hosted service terms
Recognition	**gradio-client**	Open-source client	Configured / cloud connector	Uploads an eligible lip clip to the configured Hugging Face Space and calls /predict.	`app/services/lip.py`; `requirements.txt`	Apache-2.0 client; hosted model has separate terms
Recognition	**Hugging Face Space: marshal-yash/vsr_trlrs3vox2_base**	Hosted deployment of a research lip-reading model	Current configured path	Converts a short greyscale face/mouth video into a weak text hint; cloud processing, not a local checkpoint.	`app/config.py`; `app/services/lip.py`	Hosted deployment and underlying checkpoint terms apply
Recognition	**Auto-AVSR**	Open-source research code + separate pretrained weights	Optional local path	Alternative to the Space: aligns and crops mouth frames, then decodes visual speech locally when source and checkpoint are installed.	`app/services/lip.py`; `requirements-lip.txt`	Code Apache-2.0; this project restricts the selected checkpoint to non-commercial research
Recognition	**PyTorch + TorchVision + TorchAudio**	Open-source	Optional local path	Tensor computation and model/audio/video support for local Auto-AVSR. Not needed for the current hosted path.	`requirements-lip.txt`; `app/services/lip.py`	Component-specific distribution licences
Recognition	**PyTorch Lightning + SentencePiece**	Open-source	Optional local path	Load/run Auto-AVSR's model module and support text tokenization.	`requirements-lip.txt`; `app/services/lip.py`	Apache-2.0
Recognition	**PyAV + scikit-image**	Open-source	Optional local path	Video/image processing dependencies for local Auto-AVSR.	`requirements-lip.txt`	BSD-3-Clause; PyAV FFmpeg build terms also apply
Understanding	**Groq SDK**	Open-source client	Live	Sends the text evidence packet to the hosted language model.	`app/services/fusion.py`	Apache-2.0
Understanding	**gpt-oss-20b**	Open-weight model	Current model / cloud via Groq	Uses fragments and context to propose complete sentences or a clarifying question. It is hosted remotely in this project.	`app/config.py`; `app/services/fusion.py`	Apache-2.0 model release; see official model card
Understanding	**Groq inference API**	Hosted service	Live / cloud	Runs the configured gpt-oss-20b model and returns structured JSON. Groq is the host; gpt-oss is the model.	`app/services/fusion.py`	Hosted service terms
Output	**ElevenLabs Flash v2.5**	Hosted service	Live / cloud	Turns the confirmed sentence into synthetic voice audio. Model ID: eleven_flash_v2_5.	`app/services/tts.py`	Hosted service terms
Output	**Tavus**	Hosted service	Optional / needs session	Makes an AI avatar say the confirmed sentence using an echo message. The avatar is a relay, not the sentence generator.	`app/services/avatar.py`	Hosted service terms
Output	**daily-python + @daily-co/daily-js**	Open-source clients	When avatar is used	Python joins and sends controlled echo messages; the browser receives the avatar's real-time room video/audio.	`app/services/avatar.py`; `f_new/src/components/avatar-player.tsx`	BSD-2-Clause; Daily service has separate terms
Browser APIs	**WebSocket + HTMLAudioElement**	Browser standards	Live	WebSocket delivers draft/status events; browser audio plays the local voice fallback.	`f_new/src/lib/backend.tsx`; `app/api/ws.py`	Browser APIs; not additional Python/npm dependencies

AREA	COMPONENT	TYPE	USED WHEN	WHAT IT DOES	WHERE IN CODE	LICENCE / TERMS
Browser APIs	**WebGL + Canvas + HTML video**	Browser standards	Landing only	Composite the existing video, title lettering, and foreground mask. They do not perform patient recognition.	`f_new/src/HeroScene.tsx`	Browser APIs
Browser APIs	**SpeechSynthesis**	Browser API	Landing preview only	Reads a sample sentence on explicit click using the device voice. Separate from the real ElevenLabs app pipeline.	`f_new/src/SootheStudio.tsx`	Browser-provided API/voices
Our logic	**ConsentRegistry + TurnMachine**	Project code	Live / local	Gate inputs by permission and coordinate capture, suggestion, clarification, editing, and speaking states.	`app/core/consent.py; app/core/turn_machine.py`	Repository MIT
Our logic	**FragmentCollector + deterministic cleanup**	Project code	Live / local	Keep word order and pauses; flag repeats, fillers, stutters, and weak evidence; produce the cleaned draft.	`app/core/fragments.py`	Repository MIT
Our logic	**Conversation + Phrasebook + ProfileTerms**	Project code	Live / local	Keep recent speaker-labelled context in memory; optionally save personalized words/sentences to local JSON after consent.	`app/core/conversation.py; app/core/phrasebook.py; data/`	Repository MIT
Our logic	**FusionService + evidence validator**	Project code + Pydantic	Live / local	Build model input, check output coverage, and reject unsupported added words. This is orchestration, not a separate trained model.	`app/services/fusion.py; app/schemas.py`	Repository MIT + Pydantic MIT
Our logic	**ConfirmationRegistry**	Project code	Live / local	Issue an expiring, one-time ID bound to the selected text before speech output can run.	`app/core/confirmation.py`	Repository MIT
Our logic	**EventBus + HealthRegistry + Metrics**	Project code	Live / local	Publish fragment/draft events, report availability, and record stage timings and counters.	`app/core/events.py; app/core/health.py; app/core/timing.py`	Repository MIT
Verification	**pytest + fake adapters**	Open-source test runner + project code	Tests only	Test capture, permissions, fragments, model-output checks, confirmation, and fallback without using live devices or APIs.	`tests/; app/adapters/fakes.py`	pytest MIT; adapters repository MIT
Verification	**Playwright**	Open-source	Browser tests	Check navigation, responsive layouts, video controls, themes, and browser errors.	`f_new/scripts/check-browser.mjs; f_new/scripts/verify-in-app.mjs`	Apache-2.0
Verification	**Ruff**	Open-source	Development only	Checks Python code style and common mistakes.	`requirements-dev.txt; pyproject.toml`	MIT
Verification	**Custom evaluation harness**	Project code	Evaluation only	Measures word error rate, intent proxies, latency, and different input combinations. Synthetic scenarios are labelled.	`evaluation/`	Repository MIT
Asset preparation	**rembg / U²-Net + Pillow + SciPy**	Open-source tools + separate model	Offline asset preparation	Create foreground masks for the supplied landing video. Not the user's live camera or lip-reading pipeline.	`f_new/scripts/prepare-media.py`	Check tool/model licences in their distributions
Asset preparation	**FFmpeg + OpenCV**	Open-source tools	Offline asset preparation	Encode the packed hero video; OpenCV removes a fixed symbol from supplied frames. No patient-media processing in these scripts.	`f_new/scripts/prepare-media.py; f_new/scripts/remove-symbol.py`	FFmpeg build-dependent LGPL/GPL; OpenCV Apache-2.0
Asset preparation	**DM Sans + Cormorant Garamond**	Open font assets	Landing/app assets	Self-hosted typography; local font licence files ship with the frontend.	`f_new/public/fonts/`	See bundled OFL licence files

The details that matter.

▼ Partner: same recognition, different purpose

A partner question is recorded through the same local microphone and Scribe service. It is saved as speaker-labelled conversation context. The patient's next words can use that context. Example: "Water or tea?" + "water … cold" → "I would like cold water, please." Speaker roles are selected explicitly; there is no automatic diarization.

▼ Confirmation, auto-say, and the output ladder

Our validator checks the model response before review. A single-use confirmation ID binds the exact text. Optional auto-say offers a 3-second cancel/edit window for eligible candidates; lip-only evidence is excluded. Edited wording is shown back. Output tries Tavus audio echo, then text echo, then local voice audio/subtitles. An earlier successful tier stops later tiers from speaking too.

▼ Current deployment, not a claim of perfect availability

Current configuration: real mode, gpt-oss-20b through Groq; scribe_v2 and eleven_flash_v2_5 through ElevenLabs; HF lip Space marshal-yash/vsr_trlrs3vox2_base; MacBook Air Microphone. Backend: 8765. Frontend: 5173. Services still depend on permissions, credentials, provider availability, and optional avatar sessions.

▼ Lip recognition: the actual data handling

The currently configured hosted path writes a temporary MP4, uploads it to the HF Space, and deletes it in a finally block. It is therefore not entirely memory-only and not entirely local. Audio is also sent to ElevenLabs for recognition. Only text and derived evidence are sent to Groq. Some older README privacy wording predates this path.

▼ Landing demo and video tools are separate

The landing studio uses labelled examples and optional browser SpeechSynthesis. The cinematic hero uses WebGL/Canvas/video. Offline rembg/U²-Net, Pillow, SciPy, OpenCV, and FFmpeg preparation builds those visual assets; it does not perform patient recognition.

▼ Licensing and primary sources

Official gpt-oss-20b model card · Auto-AVSR source · Repository licence audit. gpt-oss is an open-weight release under Apache-2.0, but runs through hosted Groq here. SDK licences do not license cloud services. MediaPipe toolkit and model assets have distinct terms; the repository audit has not confirmed the downloaded model-asset licences. The selected Auto-AVSR checkpoint is restricted to non-commercial research in this project's records. This map is not a fresh legal audit.